# Trabalho Prático 1 – Indexação Geoespacial e Geometria Computacional Híbrida

Joseph Patrick Alves de Moura
2024005548

20 de outubro de 2026

## Conteúdo

**1. Introdução** | 3

**2. Módulos Implementados** | 3

2.1. Tratamento de Dados (tratamento_dados.py) | 3

2.2. Codificação Geohash e Vizinhança (geohash.py) | 4

2.3. Árvore de Prefixos Compacta (trie.py) | 4

2.4. Simulador de Entregadores (entregadores.py) | 5

2.5. Interface Interativa e Tempo Estimado (app.py) | 5

2.6. Fluxo de Execução entre os Módulos | 6

2.7. Análise Geral de Complexidade | 6

**3. Estratégias de Robustez e Testes** | 7

**4. Análise Experimental** | 8

**5. Conclusão** | 9

**6. Bibliografia** | 10

## 1. Introdução

A otimização de consultas espaciais em tempo real é um desafio central em sistemas modernos de geolocalização, como plataformas de entrega de refeições e aplicativos de transporte urbano. Em uma cidade como Belo Horizonte, identificar rapidamente quais entregadores de uma grande frota estão próximos a um restaurante exige estruturas de dados que evitem o cálculo exaustivo de distâncias para todos os veículos em operação.

O objetivo principal deste trabalho é implementar e analisar um motor de busca e monitoramento geoespacial híbrido — denominado UFood — capaz de integrar estruturas de dados para processamento de strings e algoritmos de geometria computacional. A solução combina o uso da codificação Geohash (em Base32 com até 8 níveis de precisão) com o armazenamento e recuperação por prefixos em uma Árvore Trie Compacta implementada do zero, permitindo podar o espaço de busca em tempo proporcional ao tamanho do prefixo e ao número de candidatos locais, O(m + k), antes do refinamento geométrico final.

Para validar o sistema em um cenário realista, o programa foi desenvolvido em Python utilizando a base pública de atividades econômicas da Prefeitura de Belo Horizonte (PBH) e a malha viária real do OpenStreetMap, usando a biblioteca osmnx. A solução é acompanhada de uma análise experimental que avalia os custos computacionais de indexação e consulta sob diferentes tamanhos de frota (N) e raios de atendimento, comprovando a eficácia prática da indexação por prefixo.

## 2. Módulos Implementados

O sistema foi desenvolvido em Python seguindo uma arquitetura modular no diretório src/, separando a manipulação da base de dados geográficos, as estruturas de dados centrais, a geração sintética de dados e a camada de visualização. Abaixo são descritas as responsabilidades gerais e as decisões de implementação de cada módulo.

### 2.1. Tratamento de Dados (tratamento_dados.py)

Este módulo lê, filtra e padroniza a base de atividades econômicas da Prefeitura de Belo Horizonte, escolhida como fonte dos dados por ser um arquivo local e estável, o que dispensa conexão com APIs durante a execução. A seleção mantém os estabelecimentos que atendam a pelo menos um dos critérios: código CNAE entre 5611203, 5611202, 5611205 e 5620104, ou nome fantasia contendo as palavras BAR, LANCHONETE ou RESTAURANTE. A união dos critérios amplia a cobertura, mas também pode incluir registros de outros ramos. Os tipos de logradouro, codificados pela prefeitura (RUA, AVE, PCA, ROD etc.), são expandidos para o nome completo (Rua, Avenida, Praça, Rodovia etc.). O endereço é montado a partir dos campos de logradouro, número e bairro e registros sem coordenada ou sem nome fantasia são descartados. As coordenadas estão no sistema plano UTM e são convertidas para latitude e longitude com a biblioteca pyproj, porque o geohash e a distância de Haversine trabalham em graus.

### 2.2. Codificação Geohash e Vizinhança (geohash.py)

Este módulo transforma latitude e longitude em geohashes de 8 caracteres, na base 32. Cada coordenada gera 40 bits, sendo 20 de longitude e 20 de latitude, intercalados bit a bit com a longitude

primeiro. A cada bit, o intervalo do eixo é dividido ao meio, e o bit indica em qual metade o ponto está. Pontos geograficamente próximos podem cair em células com prefixos diferentes quando estão perto de uma borda de divisão. Para resolver isso, a busca consulta a célula do restaurante e suas 8 vizinhas imediatas. O prefixo usado é o maior cuja célula ainda tem tamanho maior ou igual ao raio de busca. Por exemplo, para um raio de 2 km, o prefixo tem 5 caracteres, com células de 4,6 km por 4,9 km. Como cada lado da célula é pelo menos igual ao raio, o círculo de atendimento fica inteiro dentro das 9 células consultadas, e a filtragem geométrica final não perde candidatos.

A escolha do prefixo está implementada em escolher_precisao, no módulo app.py.

### 2.3. Árvore de Prefixos Compacta (trie.py)

Este módulo implementa do zero as classes Node e Trie, a estrutura central de indexação do trabalho, sem bibliotecas externas. Para a Trie, foram implementados métodos de inserção, busca por palavra e busca por prefixo, onde este último retorna todos os geohashes inseridos na estrutura que começam com determinado prefixo. Por ser uma Trie Compacta, cada nó guarda um trecho de string, o que compacta caminhos sem ramificação. Um nó com um filho "**$**" marca o fim de uma palavra. Uma folha também representa o fim de uma palavra e, por isso, não recebe "**$**". Para gerenciar a frota, a Trie guarda apenas geohashes distintos. Um dicionário auxiliar associa cada geohash à lista de entregadores que caíram naquela célula, o que evita perder entregadores que estejam na mesma posição.

A busca por prefixo desce até o nó correspondente e coleta todas as palavras da subárvore, incluindo folhas e nós marcados com "**$**". Como as chaves têm no máximo 8 caracteres, cada busca custa O(m + k), com m sendo o tamanho do prefixo e k o número de resultados. Uma consulta faz nove buscas, uma para a célula do restaurante e uma para cada vizinha, então o custo total é a soma dessas nove buscas.

### 2.4. Simulador de Entregadores (entregadores.py)

Este módulo concentra a geração dos dados sintéticos da frota. A classe Entregador guarda ID, nome fictício (gerado coma biblioteca Faker), velocidade entre 10 e 30 km/h, coordenadas e geohash de 8 caracteres. Para simular a ociosidade real de aplicativos de entrega, os N entregadores são divididos igualmente entre os restaurantes da base. Quando N não é múltiplo do número de restaurantes, o restante é sorteado entre eles. Cada entregador recebe uma posição sorteada de uma distribuição normal centrada no restaurante, com desvio padrão $\sigma = 700$ m. Com esse valor, cerca de 86% dos entregadores ficam a até 1,4 km do centro, o que concentra a frota perto dos restaurantes e deixa uma parcela fora do raio de 2 km.

A geração puramente matemática posicionaria entregadores em locais inacessíveis, como no meio de quarteirões, casas, etc. Por isso, depois do sorteio, cada posição é projetada no nó do grafo viário do OpenStreetMap mais próximo, usando ***osmnx.distance.nearest_nodes***. A projeção é feita em uma única chamada vetorizada para toda a frota, e a partir do nó encontrado são calculadas as coordenadas e o geohash finais.

### 2.5. Interface Interativa e Tempo Estimado (app.py)

A interface foi desenvolvida com Streamlit e Folium, escolhidos pela facilidade de integrar o mapa interativo aos controles e à tabela de resultados. A busca textual procura estabelecimentos pelo nome

fantasia ou pelo endereço, e os resultados aparecem em ordem alfabética, sem diferenciar maiúsculas de minúsculas. O raio de filtragem tem dois controles sincronizados. Um slider, de 0,1 a 30 km com passo de 0,1 km, e uma caixa numérica, de 50 a 30.000 m com passo de 50 m, que permite escolher um valor exato. Alterar um dos controles atualiza o outro. A quantidade de entregadores N é informada em uma caixa numérica, de 1 a 50.000.

O tempo estimado de chegada é calculado como ETA = (distância / velocidade) × 60, em minutos, com a velocidade individual de cada entregador sorteada entre 10 e 30 km/h. A distância é em linha reta (Haversine). Trata-se de uma estimativa simplificada, que não usa o tempo real do trajeto pelas vias da cidade.

### 2.6. Fluxo de Execução entre os Módulos

A integração entre os módulos é coordenada pelo arquivo **app.py**, em três etapas.

1. **Inicialização dos dados:** Ao abrir a aplicação, **app.py** aciona **tratamento_dados.py** para carregar, limpar e projetar a base de estabelecimentos da Prefeitura. O DataFrame resultante fica em cache. A malha viária não é carregada nesse momento.

2. **Simulação e indexação da frota:** Quando o usuário informa N e clica em "Gerar Entregadores", **app.py** chama **entregadores.py**, que distribui os N entregadores entre os restaurantes, sorteia posições com distribuição normal (σ = 700 m) e as projeta para o nó mais próximo da malha viária, carregada ou baixada via osmnx na primeira vez. Em seguida, **geohash.py** codifica cada posição em 8 caracteres na base 32. Por fim, **app.py** instancia a Trie, insere os geohashes distintos e preenche o dicionário que associa cada geohash à lista de entregadores.

3. **Consulta e filtragem:** Ao selecionar um estabelecimento e um raio, **app.py** escolhe o prefixo m. Em seguida, **geohash.py** calcula os 9 prefixos (célula central e 8 vizinhas), e a Trie retorna os candidatos de cada um via retorna_prefixo(). Para esses candidatos, app.py calcula a distância de Haversine, mantém os que estão dentro do raio, calcula o ETA e atualiza o mapa e as tabelas.

### 2.7. Análise Geral de Complexidade

De forma geral, a eficiência do sistema decorre do custo constante das operações de codificação e da poda espacial proporcionada pela árvore de prefixos. Como a precisão máxima do Geohash é fixa em P = 8 caracteres (40 bits sobre um alfabeto de 32 símbolos), tanto a codificação de uma coordenada quanto a inserção de uma chave na Trie executam passos em tempo O(P) = O(1) por entregador. Assim, a construção completa do índice para uma frota de tamanho N possui complexidade O(N).

Na etapa de consulta, o cálculo das 9 células vizinhas tem custo O(1). Para cada uma das 9 células, a descida na Trie Compacta leva tempo proporcional ao prefixo, O(m), e o percurso nas subárvores coleta os k candidatos da vizinhança em tempo O(k). O filtro geométrico de Haversine avalia exatamente esses k candidatos em tempo O(k). Portanto, a recuperação e filtragem espacial respeitam rigorosamente a complexidade assintótica O(m + k) exigida pelo enunciado, tornando o tempo de consulta independente do tamanho total N da frota espalhada pelo restante da cidade.

### 2.8. Cache e Persistência

Para evitar trabalho repetido, o sistema usa três níveis de armazenamento. A base de restaurantes é lida, filtrada e projetada uma única vez por processo, com st.cache_resource, e fica em memória nas consultas seguintes. A malha viária é baixada do OpenStreetMap apenas na primeira execução e salva em data/bh.graphml. Nas execuções seguintes, o grafo é lido do disco, o que dispensa conexão com a internet e reduz o tempo de carregamento. A frota, a Trie e o dicionário de entregadores ficam no estado da sessão e são recriados a cada clique em "Gerar Entregadores", conforme o enunciado exige.

Na seção experimental, os tempos de primeira execução (leitura do CSV e carregamento do grafo) são reportados separadamente dos tempos de consulta, que são medidos com o cache já aquecido.

## 3. Estratégias de Robustez e Testes

Para garantir a estabilidade do sistema, o desempenho com grandes frotas e a confiabilidade dos algoritmos, foram adotadas as seguintes práticas.

- **Persistência e cache de recursos:** O grafo viário de Belo Horizonte é salvo em data/bh.graphml após o primeiro download e mantido em memória com @st.cache_resource, junto à base de restaurantes. Isso evita recarregamentos nas consultas seguintes;
- **Degradação na renderização do mapa:** Quando a consulta retorna até 300 entregadores dentro do raio, o mapa exibe ícones de motocicleta. Acima disso, os filtrados passam a ser CircleMarker, mais leves para o navegador. Os descartados são sempre desenhados como círculos cinzas, sem limite, o que pode pesar em frotas muito concentradas;
- **Validação de limites e dados:** A codificação de coordenadas rejeita valores fora de [-90, 90] e [-180, 180] com erro explícito. No carregamento, registros sem coordenada são removidos, e os duplicados (mesmo nome fantasia e mesma posição) são eliminados em carregar_restaurantes();
- **Suíte de testes (tests/):** Quatro módulos com unittest cobrem a divisão de nós da Trie, a busca de prefixos inexistentes e de palavras que são prefixo de outras, a intercalação de bits com comprimentos pares e ímpares, o conjunto de 9 células vizinhas contendo a célula central, e um teste de faixa para a distância de Haversine.

## 4. Análise Experimental

Esta seção avalia o número de candidatos retornados pela Trie compacta e o tempo de resposta de cada consulta, para diferentes tamanhos de frota (N) e raios de atendimento em Belo Horizonte. As consultas foram feitas no restaurante BREIK AFONSO PENA, pela interface do UFood, com a frota gerada com projeção na malha viária. O tempo de cada consulta foi medido com time.perf_counter() e inclui a seleção do prefixo, a busca nas 9 células, o filtro de Haversine e a ordenação dos resultados. Cada linha corresponde a uma execução.

A Tabela 1 apresenta os resultados:

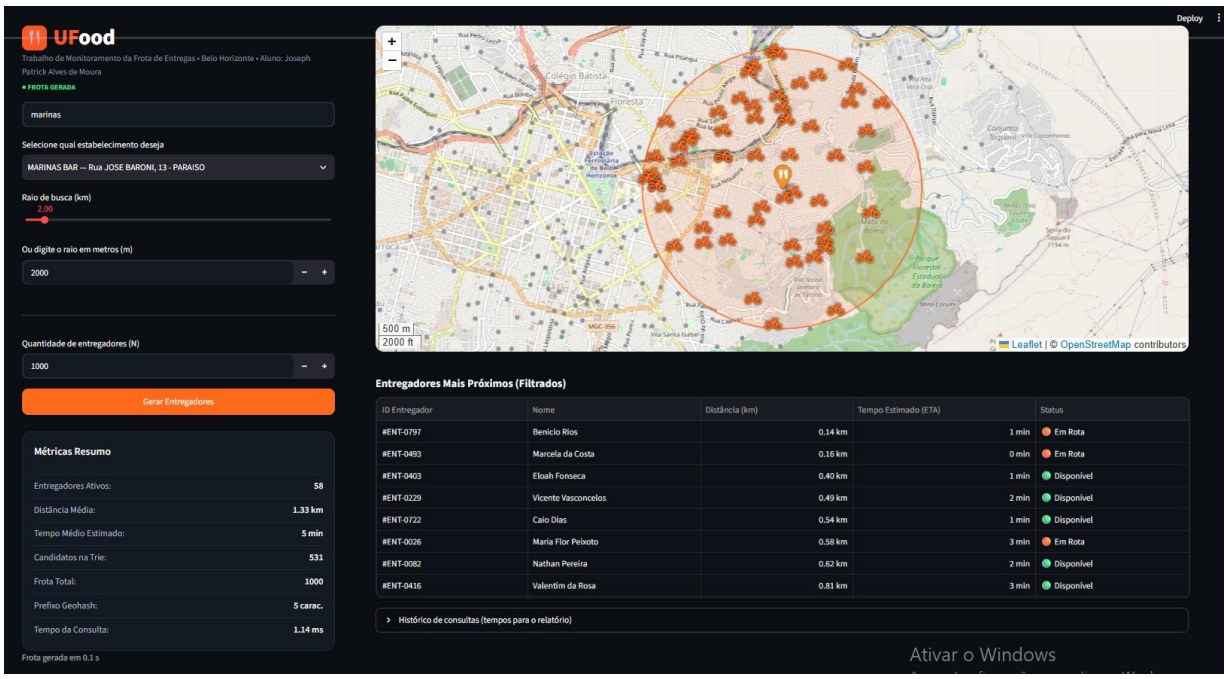
| Frota (N) | Raio (km) | Prefixo (m) | Candidatos (k) | Dentro do raio | Tempo (ms) |
|---|---|---|---|---|---|
| 1.000 | 0,1 | 7 carac. | 2 | 1 | 0,218 |
| 1.000 | 0,5 | 6 carac. | 35 | 3 | 0,281 |
| 1.000 | 2,0 | 5 carac. | 489 | 77 | 1,058 |
| 5.000 | 0,1 | 7 carac. | 15 | 2 | 0,261 |
| 5.000 | 0,5 | 6 carac. | 225 | 34 | 0,621 |
| 5.000 | 2,0 | 5 carac. | 2.650 | 393 | 5,867 |
| 10.000 | 0,1 | 7 carac. | 18 | 4 | 0,290 |
| 10.000 | 0,5 | 6 carac. | 419 | 63 | 0,890 |
| 10.000 | 2,0 | 5 carac. | 5.218 | 737 | 10,015 |
| 25.000 | 0,1 | 7 carac. | 46 | 11 | 0,361 |
| 25.000 | 0,5 | 6 carac. | 1.017 | 141 | 1,792 |
| 25.000 | 2,0 | 5 carac. | 12.934 | 1.922 | 24,823 |

Os dados mostram que o número de candidatos cresce linearmente com N, para cada raio. Para raios de 0,1 km (m = 7) e 0,5 km (m = 6), a Trie descarta mais de 95% da frota antes do filtro geométrico. Com 0,1 km, o descarte passa de 99%, e o tempo fica entre 0,22 ms e 0,36 ms. Com 0,5 km, o tempo vai de 0,28 ms a 1,79 ms, de N = 1.000 a N = 25.000.

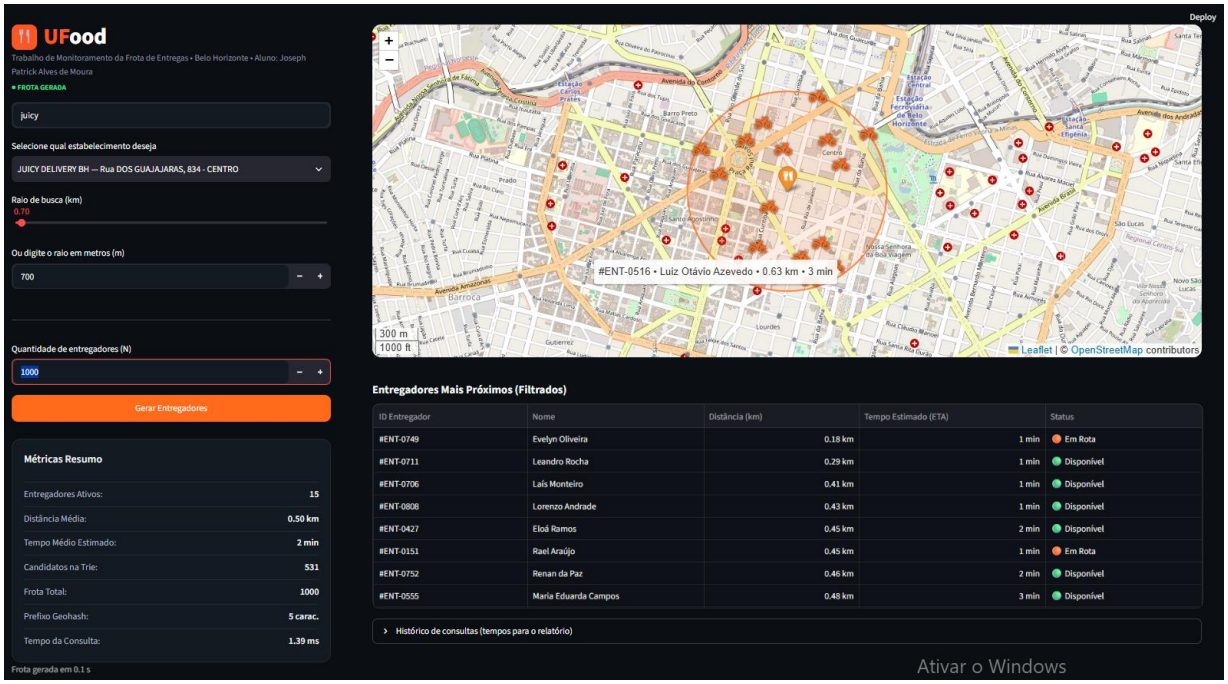
Para raio de 2 km (m = 5), a Trie descarta apenas cerca de 48% da frota, porque as 9 células somam cerca de 200 km², bem maior que o círculo de atendimento. Nesse caso, o custo é dominado pelo filtro de Haversine. O tempo sobe de 1,06 ms (N = 1.000) para 24,82 ms (N = 25.000), e o tempo por candidato fica em torno de 2 µs em todos os tamanhos, o que confirma o custo linear em k.

### 4.1. Execução de Consultas na Interface (Prints do Sistema)

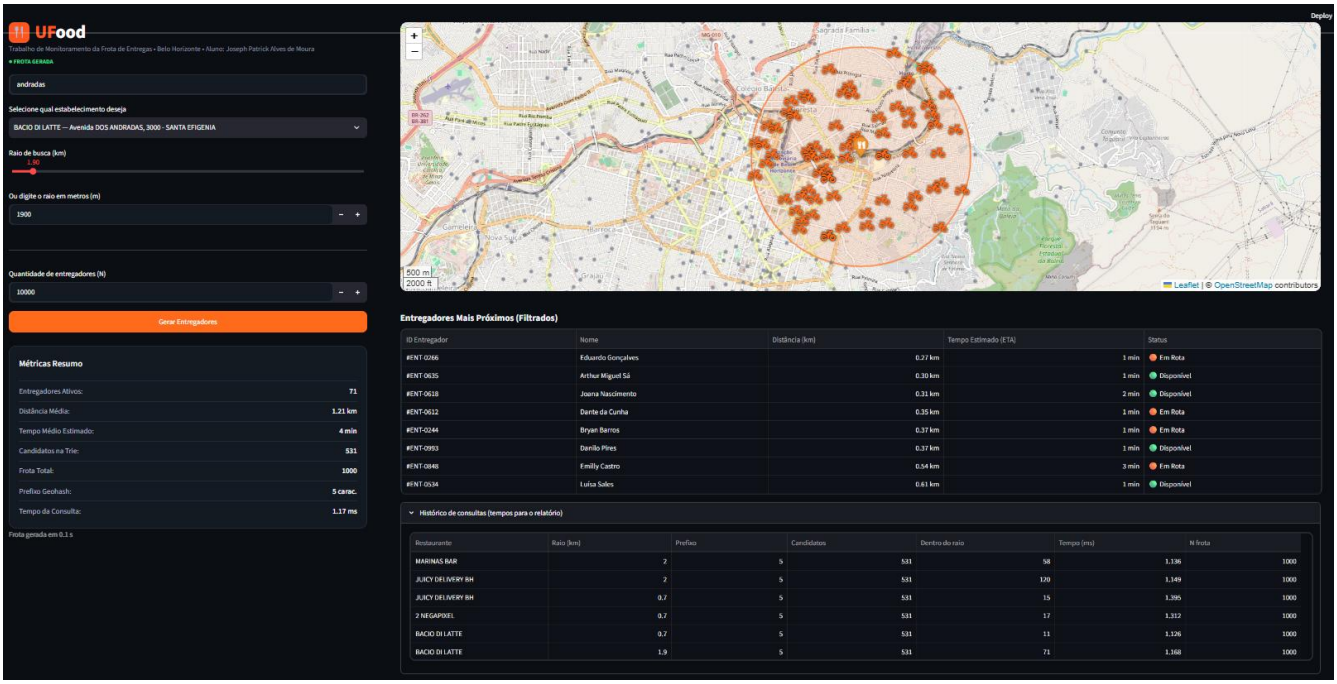
A seguir são apresentadas as capturas de tela da interface UFood demonstrando os resultados visuais das consultas realizadas, o posicionamento dos entregadores sobre as vias reais da cidade, a distinção entre entregadores filtrados (motos em laranja) e descartados (círculos cinza), além das métricas e tempos de resposta registrados:




**[Consulta 1: Restaurante consultado com frota de N = 1.000 entregadores e raio de 2,0 km (prefixo m = 5)]**




**[Consulta 2: Consulta utilizando raio reduzido (0,5 km)]**




**[Consulta 3: Consulta com frota elevada (ex.: N = 10.000 entregadores) e a seção 'Histórico de consultas']**

## 5. Conclusão

Este trabalho abordou a indexação e o monitoramento geoespacial de uma frota de entregadores em Belo Horizonte, por meio do sistema UFood. A solução foi implementada em Python, combinando a codificação Geohash em Base32 com uma Árvore de Prefixos Compacta construída do zero, e a filtragem final pela distância de Haversine.

A análise de complexidade e os experimentos são consistentes com o custo O(m + k) da busca por prefixo nas 9 células adjacentes. A busca não percorre a frota inteira, e a varredura nas células vizinhas resolve o problema de borda antes da filtragem geométrica. O poder de poda da Trie depende do raio. Para 0,1 km e 0,5 km, ela descartou entre 95% e 99,8% da frota antes do filtro, e os tempos ficaram abaixo de 2 ms para frotas de até 25.000 entregadores. Para 2 km, a poda é menor, cerca de 48%, porque as 9 células somam uma área muito maior que o círculo de atendimento. Nesse caso, o custo passa a ser dominado pelo filtro de Haversine, e o tempo cresce linearmente com o número de candidatos, chegando a 24,8 ms com 25.000 entregadores.

O projeto também permitiu aplicar conceitos da disciplina em um sistema completo. Foram integradas estruturas de dados para strings, algoritmos de geometria computacional, reprojeção da base da Prefeitura de UTM para latitude e longitude e restrição viária sobre o grafo do OpenStreetMap.

Há limitações que devem ser consideradas. Os tempos foram medidos em uma única execução por consulta, e os cenários foram testados com até 25.000 entregadores, sem o cenário de 50.000 previsto originalmente. A distância é calculada em linha reta, e o tempo estimado de chegada é uma aproximação que não usa o tempo real do trajeto pelas vias. A interface foi feita com Streamlit e Folium, e não com as bibliotecas recomendadas no enunciado. Como trabalhos futuros, seriam úteis o

cálculo de rotas reais sobre o grafo, a remoção de entregadores da Trie em tempo real e a medição do desempenho com frotas maiores e em mais restaurantes.

## 6. Bibliografia


- Vimieiro, Renato. (2026). Notas de aula e materiais da disciplina DCC207 – Algoritmos 2. Departamento de Ciência da Computação, Universidade Federal de Minas Gerais (UFMG). Belo Horizonte.
- Niemeyer, Gustavo. (2008). Geohash: Geocoding system based on hierarchical spatial data structure. Disponível em: http://geohash.org.
- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. Computers, Environment and Urban Systems, 65, 126-139.
- Prefeitura Municipal de Belo Horizonte (PBH). Portal de Dados Abertos – Base de Atividades Econômicas. Belo Horizonte.